

COMPTE RENDU DE TRAVAUX PRATIQUES

TP n°3

Analyse et Récupération d'un Signal Binaire Bruité

Modulation FSK – Filtrage FIR – Démodulation par enveloppe



Réalisé par : Briksine Aya

Encadré par : Tinhinane MEHDI

Date : Mai 2025

Table des matières

1	Introduction et Rappels Théoriques	3
1.1	Contexte	3
1.2	Chaîne de traitement mise en œuvre	3
2	Question 1 – Affichage du signal bruité	3
2.1	Rappel et objectif	4
2.2	Code MATLAB	4
2.3	Résultat et interprétation	4
3	Question 2 – Analyse spectrale	5
3.1	Rappel et objectif	5
3.2	Code MATLAB	5
3.3	Résultat et interprétation	6
4	Question 3 – Filtrage passe-bande	6
4.1	Rappel et objectif	6
4.2	Code MATLAB	6
4.3	Résultat et interprétation	8
5	Question 4 – Détection d’enveloppe	8
5.1	Rappel et objectif	8
5.2	Code MATLAB	9
5.3	Résultat et interprétation	9
6	Question 5 – Reconstruction et taux d’erreur	10
6.1	Rappel et objectif	10
6.2	Code MATLAB	10
6.3	Résultat	11
7	Conclusion	12

Table des figures

1	Signal bruité – 2 premières secondes	4
2	Spectre de fréquence du signal bruité	6
3	Signaux filtrés autour de f_0 (haut) et f_1 (bas) – 8 premières secondes	8
4	Enveloppes lissées de f_0 (haut) et f_1 (bas) – 8 premières secondes	9
5	Message original (rouge) et message reconstruit (bleu) – 8 premières secondes . .	11

Introduction et Rappels Théoriques

Contexte

Ce travail pratique porte sur l'analyse et la récupération d'un signal binaire bruité transmis par modulation de fréquence.

Rappel – Modulation FSK (*Frequency Shift Keying*)

La modulation FSK consiste à coder l'information binaire en faisant varier la fréquence du signal porteur. Pour un système FSK binaire :

$$s(t) = A \cos(2\pi f_i t), \quad f_i = \begin{cases} f_0 & \text{si bit} = 0 \\ f_1 & \text{si bit} = 1 \end{cases}$$

Le signal reçu est une succession de sinusoïdes à f_0 ou f_1 , corrompue par un bruit additif. La démodulation consiste à identifier, pour chaque intervalle bit, quelle fréquence est présente.

Chaîne de traitement mise en œuvre

La démodulation FSK suit les étapes suivantes :

1. **Visualisation temporelle** du signal bruité.
2. **Analyse spectrale** pour localiser f_0 et f_1 .
3. **Filtrage passe-bande FIR** pour isoler chaque composante.
4. **Détection d'enveloppe** par valeur absolue et lissage.
5. **Décision bit à bit** par comparaison des enveloppes et calcul du BER.

Paramètre	Valeur
Fréquence d'échantillonnage F_s	1999 Hz
Longueur du message	500 bits
Longueur du signal bruité	999 500 échantillons
Durée totale	500 secondes
Échantillons par bit (N_{bit})	1999
Fréquence f_0	≈ 277 Hz (bit 1)
Fréquence f_1	≈ 522 Hz (bit 0)

TABLE 1 – Paramètres du signal `signal_28032023_1.mat`

Question 1 – Affichage du signal bruité

Rappel et objectif

Rappel – Représentation temporelle

Afficher un signal dans le domaine temporel permet d'observer son amplitude, sa stationnarité apparente et l'impact du bruit. On construit l'axe temporel $t = n/F_s$ où n est l'indice de l'échantillon.

L'objectif ici est de visualiser les 2 premières secondes du signal bruité afin d'avoir un aperçu de sa structure temporelle.

Code MATLAB

Listing 1 – Affichage du signal bruité sur 2 secondes

```
1 t = (0:length(signal_bruit)-1) / Fs;  
2 plot(t(t <= 2), signal_bruit(t <= 2))  
3 xlabel('Temps (s)')  
4 ylabel('Amplitude')  
5 title('Signal bruité - 2 premières secondes')
```

Résultat et interprétation

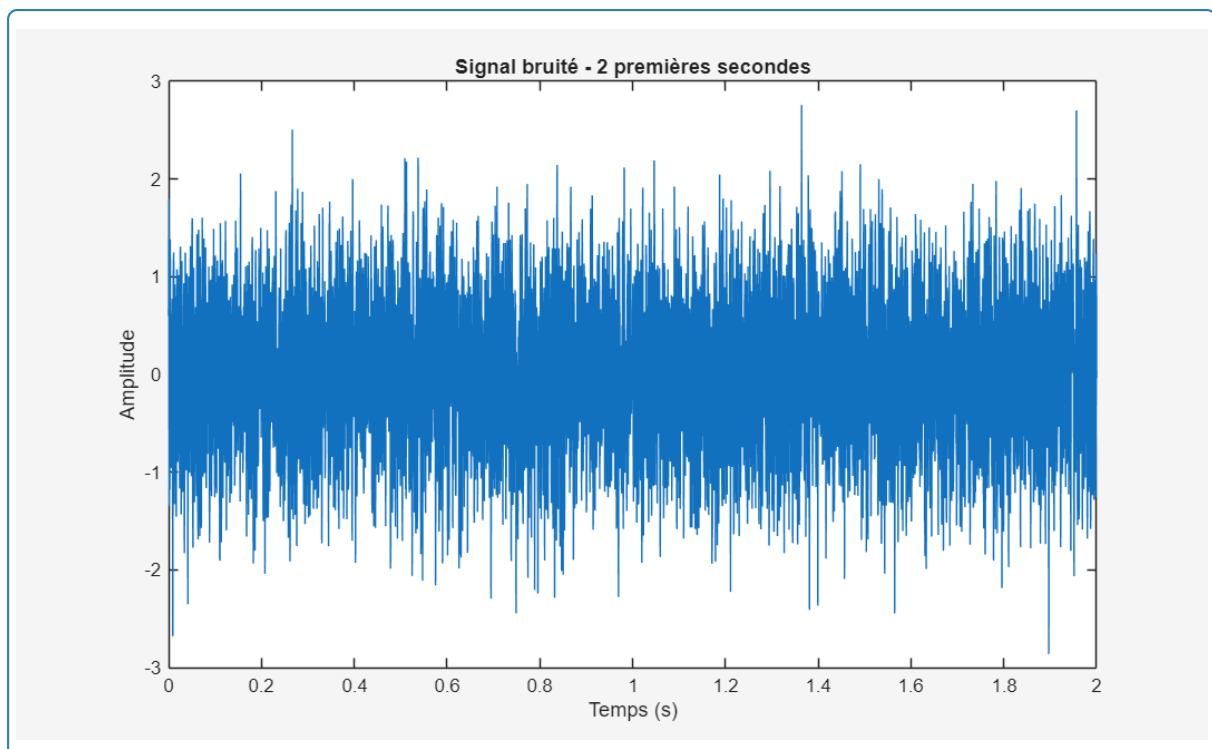


FIGURE 1 – Signal bruité – 2 premières secondes

Le signal bruité présente une amplitude oscillant entre -3 et $+3$, avec un aspect chaotique dû au bruit additif. Il est impossible à ce stade d'identifier visuellement le message binaire sous-jacent. L'amplitude reste néanmoins bornée, ce qui est cohérent avec un signal FSK bruité.

Question 2 – Analyse spectrale

Rappel et objectif

Rappel – Transformée de Fourier et spectre

La Transformée de Fourier Discrète (TFD) décompose un signal en ses composantes fréquentielles. Pour un signal FSK, le spectre présente des pics aux fréquences f_0 et f_1 . La fréquence normalisée ω (en rad/échantillon) est liée à la fréquence physique f (en Hz) par :

$$f = \frac{\omega \cdot F_s}{2\pi}$$

L'outil `signalAnalyzer` de MATLAB permet de visualiser le spectre de puissance et de lire directement les fréquences normalisées des pics.

Code MATLAB

Listing 2 – Analyse spectrale du signal bruité

```
1 signalAnalyzer(signal_bruite)
```

Résultat et interprétation

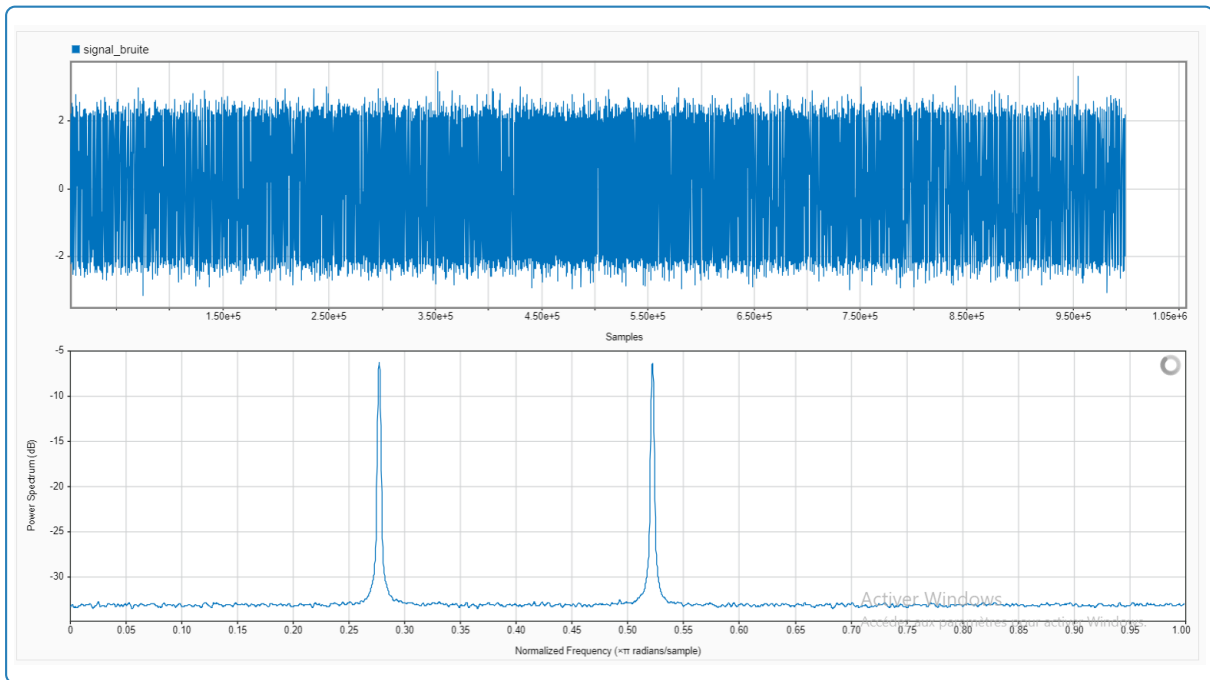


FIGURE 2 – Spectre de fréquence du signal bruité

Le spectre révèle **deux pics très nets** :

- $\omega_0 \approx 0,27\pi$ rad/échantillon $\Rightarrow f_0 \approx 277$ Hz
- $\omega_1 \approx 0,52\pi$ rad/échantillon $\Rightarrow f_1 \approx 522$ Hz

Ces deux fréquences correspondent bien aux deux symboles de la modulation FSK.

Question 3 – Filtrage passe-bande

Rappel et objectif

Rappel – Filtre FIR passe-bande

Un filtre à Réponse Impulsionnelle Finie (FIR) est conçu pour laisser passer une bande de fréquences et atténuer le reste. La fonction `fir1` de MATLAB génère les coefficients b du filtre selon l'ordre et la bande normalisée $[f_{low}, f_{high}]/(F_s/2)$. L'application du filtre se fait par convolution discrète : `filter(b, 1, signal)`.

On applique deux filtres passe-bande centrés respectivement sur f_0 et f_1 , avec une largeur de bande de ± 50 Hz.

Code MATLAB

Listing 3 – Filtrage passe-bande FIR – 8 premières secondes

```
1 w0 = 0.27*pi; w1 = 0.52*pi;
2 ordre = 50;
3 f0 = w0*Fs/(2*pi); % ~277 Hz
4 f1 = w1*Fs/(2*pi); % ~522 Hz
5
6 bande0 = [(f0-50)/(Fs/2), (f0+50)/(Fs/2)];
7 b0 = fir1(ordre, bande0, 'bandpass');
8 signal_f0 = filter(b0, 1, signal_bruite);
9
10 bande1 = [(f1-50)/(Fs/2), (f1+50)/(Fs/2)];
11 b1 = fir1(ordre, bande1, 'bandpass');
12 signal_f1 = filter(b1, 1, signal_bruite);
13
14 % Affichage sur les 8 premières secondes
15 figure;
16 subplot(2,1,1)
17 plot(t(t <= 8), signal_f0(t <= 8))
18 title('Signal filtre f0 - bit 0')
19 xlabel('Temps (s)'); ylabel('Amplitude')
20
21 subplot(2,1,2)
22 plot(t(t <= 8), signal_f1(t <= 8))
23 title('Signal filtre f1 - bit 1')
24 xlabel('Temps (s)'); ylabel('Amplitude')
```

Résultat et interprétation

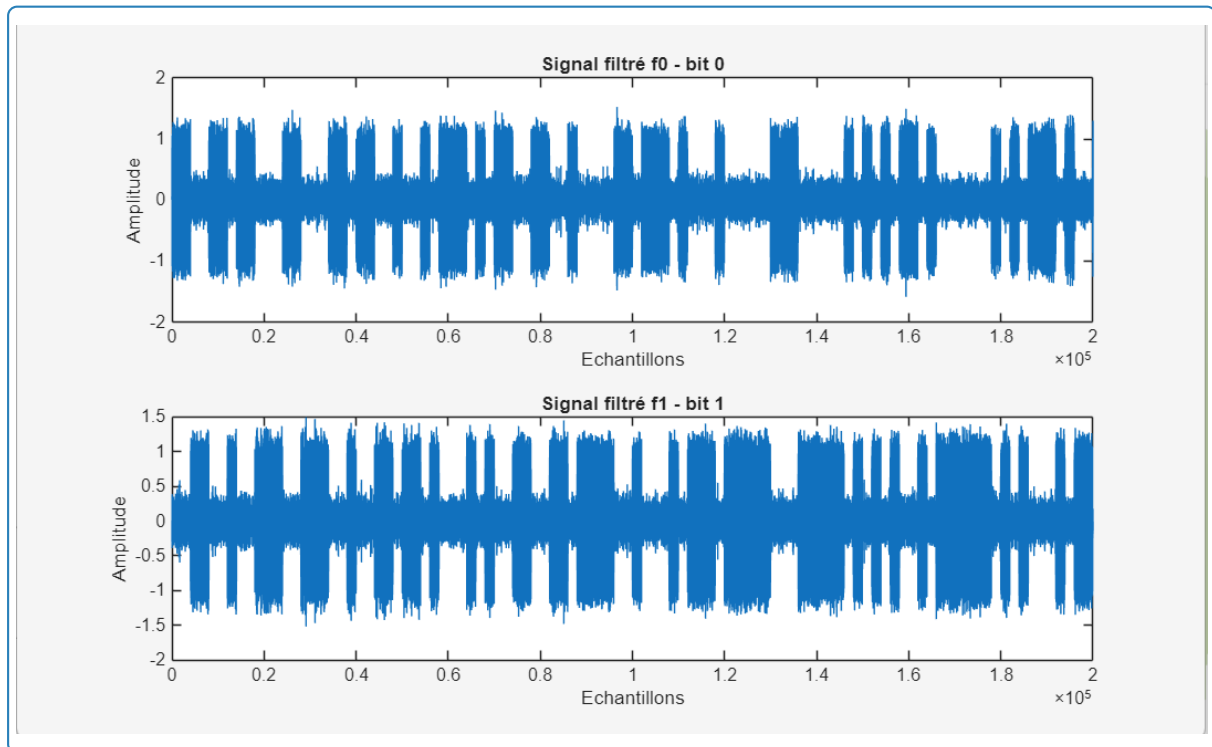


FIGURE 3 – Signaux filtrés autour de f_0 (haut) et f_1 (bas) – 8 premières secondes

Les deux signaux sont **complémentaires** : lorsque l'amplitude de f_0 est forte, celle de f_1 est faible, et inversement. Ce comportement confirme bien la nature FSK du signal : à chaque instant, une seule fréquence est active.

Question 4 – Détection d'enveloppe

Rappel et objectif

Rappel – Enveloppe et lissage

L'enveloppe d'un signal sinusoïdal représente l'amplitude instantanée de ses oscillations. Elle est estimée par la **valeur absolue** du signal filtré, puis lissée par une **moyenne glissante** de taille N_{bit} (nombre d'échantillons par bit) pour éliminer les oscillations rapides :

$$\text{env}[n] = \frac{1}{N_{bit}} \sum_{k=0}^{N_{bit}-1} |s[n-k]|$$

On obtient ainsi une estimation de l'énergie locale du signal sur chaque bit.

Code MATLAB

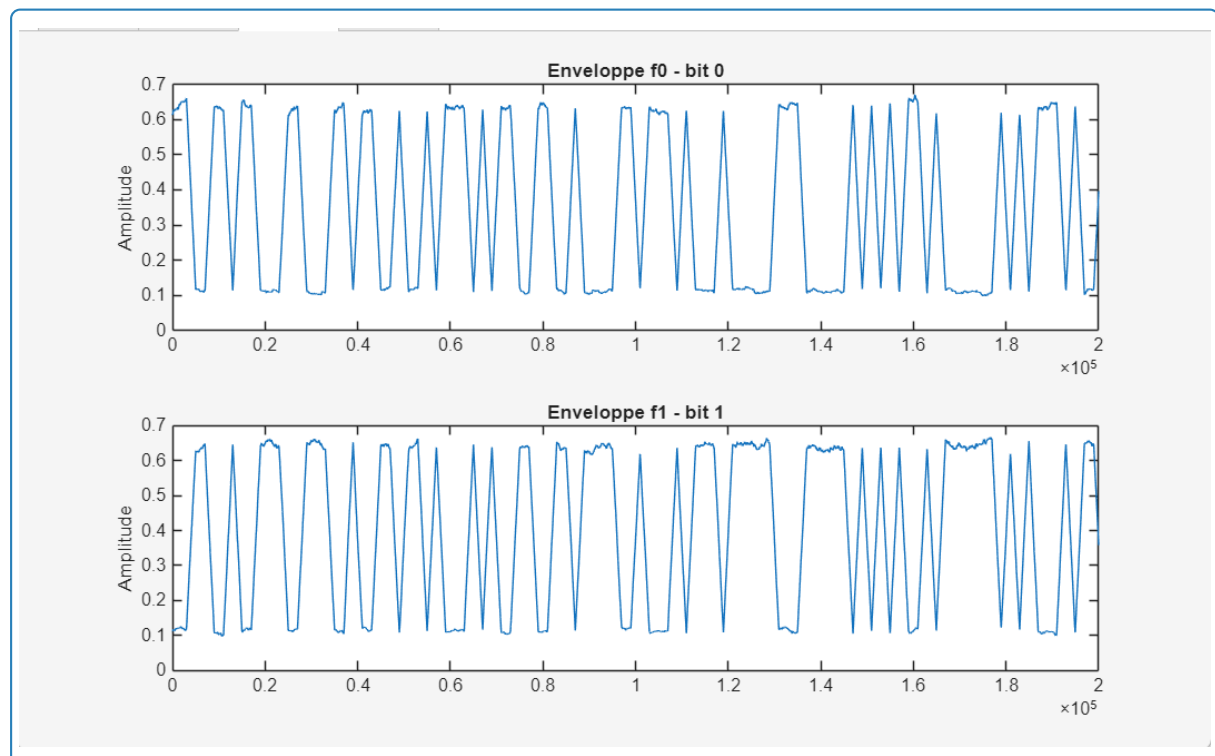
Listing 4 – Calcul et affichage de l'enveloppe – 8 premières secondes

```

1 N_bit = floor(length(signal_bruite) / length(message));
2 env_f0 = movmean(abs(signal_f0), N_bit);
3 env_f1 = movmean(abs(signal_f1), N_bit);
4
5 figure;
6 subplot(2,1,1)
7 plot(t(t <= 8), env_f0(t <= 8))
8 title('Enveloppe f0 - bit 0')
9 xlabel('Temps (s)'); ylabel('Amplitude')
10
11 subplot(2,1,2)
12 plot(t(t <= 8), env_f1(t <= 8))
13 title('Enveloppe f1 - bit 1')
14 xlabel('Temps (s)'); ylabel('Amplitude')

```

Résultat et interprétation

FIGURE 4 – Enveloppes lissées de f_0 (haut) et f_1 (bas) – 8 premières secondes

Les enveloppes présentent des créniaux bien définis et stables, alternant entre une valeur haute et une valeur basse. Les deux enveloppes sont parfaitement complémentaires : quand env_{f_0} est haute, env_{f_1} est basse, ce qui permet une décision fiable bit à bit.

Question 5 – Reconstruction et taux d'erreur

Rappel et objectif

Rappel – Décision et BER

La décision bit à bit consiste à comparer, pour chaque intervalle de N_{bit} échantillons, la moyenne des deux enveloppes :

$$\hat{b}_i = \begin{cases} 1 & \text{si } \overline{\text{env}_{f_0}}^{(i)} > \overline{\text{env}_{f_1}}^{(i)} \\ 0 & \text{sinon} \end{cases}$$

Le **Taux d'Erreur Binaire** (BER) mesure la qualité de la démodulation :

$$\text{BER} = \frac{\text{nombre de bits erronés}}{\text{nombre total de bits}} \times 100\%$$

Code MATLAB

Listing 5 – Décision bit à bit et affichage – 8 premières secondes

```

1 message_reconstruit = zeros(1, length(message));
2 for i = 1:length(message)
3     debut = (i-1)*N_bit + 1;
4     fin    = i * N_bit;
5     if mean(env_f0(debut:fin)) > mean(env_f1(debut:fin))
6         message_reconstruit(i) = 1;
7     else
8         message_reconstruit(i) = 0;
9     end
10 end
11
12 % Affichage des N premiers bits (dans 8 secondes)
13 N_bits_8s = floor(8 * Fs / N_bit);
14 figure;
15 subplot(2,1,1)
16 stem(1:N_bits_8s, message(1:N_bits_8s), 'r')
17 title('Message original (8 premières secondes)')
18 xlabel('Index du bit'); ylabel('Bit'); ylim([-0.2 1.2])
19
20 subplot(2,1,2)
```

```

21 stem(1:N_bits_8s, message_reconstruit(1:N_bits_8s), 'b')
22 title('Message reconstruit (8 premières secondes)')
23 xlabel('Index du bit'); ylabel('Bit'); ylim([-0.2 1.2])
24
25 % Taux d'erreur sur tout le message
26 erreurs = sum(message_reconstruit ~= message);
27 taux_erreur = (erreurs / length(message)) * 100;
28 fprintf('Bits erronés : %d / %d\n', erreurs, length(message));
29 fprintf('Taux d erreur : %.2f %%\n', taux_erreur);

```

Résultat

Résultat final

Nombre de bits erronés : 0 / 500

Taux d'erreur : 0,00 %

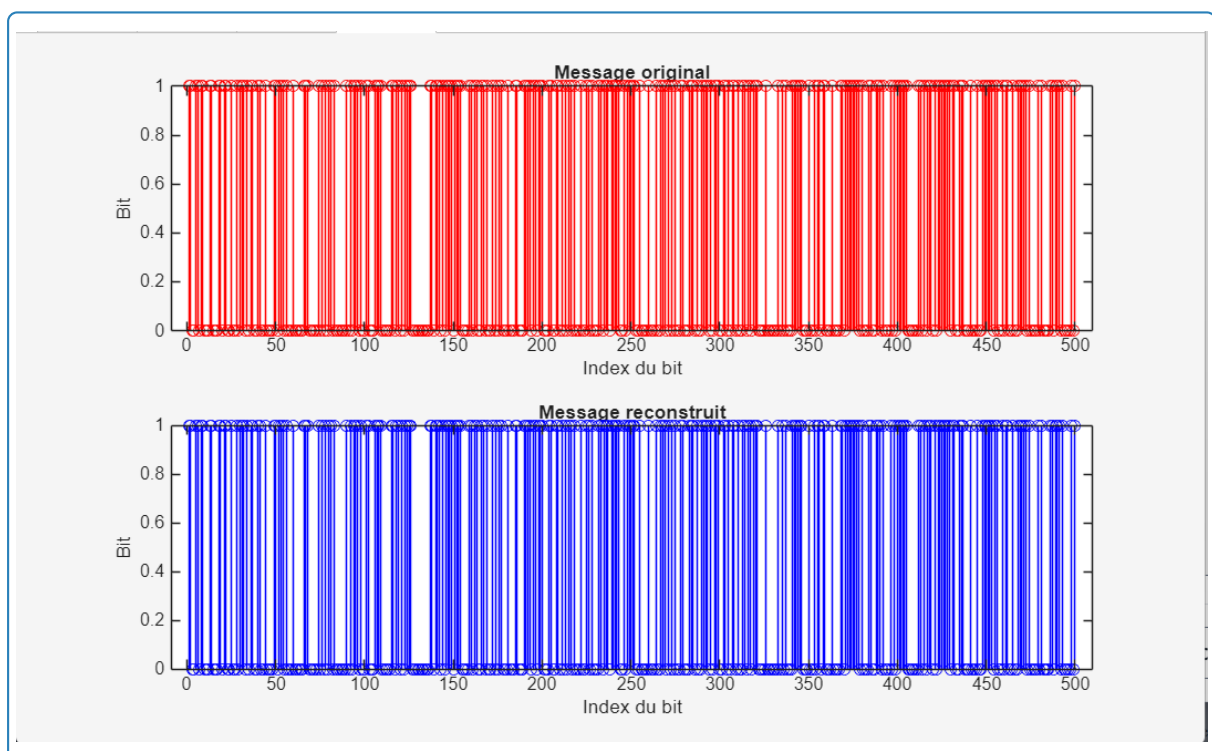


FIGURE 5 – Message original (rouge) et message reconstruit (bleu) – 8 premières secondes

Conclusion

Ce TP a permis de mettre en œuvre une chaîne complète de démodulation FSK :

- L'analyse spectrale a identifié avec précision $f_0 \approx 277$ Hz et $f_1 \approx 522$ Hz à partir du spectre de puissance.
- Le filtrage passe-bande FIR (ordre 50, largeur ± 50 Hz) a séparé efficacement les deux composantes fréquentielles.
- La détection d'enveloppe par valeur absolue et moyenne glissante a produit des créneaux nets permettant une décision fiable.
- Le taux d'erreur binaire est **nul** sur l'ensemble des 500 bits.

Remarque importante

La convention d'encodage était inversée : f_0 (277 Hz) code le bit 1 et f_1 (522 Hz) code le bit 0. Cette inversion a été détectée grâce à la comparaison entre message original et reconstruit, et corrigée dans la règle de décision.